PLAN DE IMPLEMENTACIÓN Y MIGRACIÓN A AWS

Platzi Wallet

Documento técnico de continuidad del diseño del sistema

Versión	1.0
Estado	Borrador para entrega académica-profesional
Enfoque arquitectónico	Monolito modular cloud-ready
Plataforma objetivo	Amazon Web Services (AWS)
Stack principal	Laravel + MySQL + Redis + S3
Propósito	Definir cómo construir y migrar progresivamente Platzi Wallet hacia una implementación real, operable y preparada para endurecimiento de seguridad.

Resumen ejecutivo. Esta etapa convierte el diseño arquitectónico previo y la selección de AWS de la comparativa cloud en un plan concreto de implementación. La estrategia propuesta evita un salto prematuro a microservicios completos y adopta un enfoque predominantemente Replatform: mantener la lógica de negocio en Laravel, desplegarla en contenedores administrados y trasladar datos, caché, archivos y mensajería hacia servicios gestionados de AWS. El resultado esperado es una base estable, operable y preparada para reforzar seguridad, observabilidad y automatización en la siguiente fase.

Contenido

1. 1. Propósito y relación con las etapas anteriores
2. 2. Decisión de arquitectura y estrategia de migración
3. 3. Arquitectura objetivo en AWS
4. 4. Fase 1: Assess (evaluación)
5. 5. Fase 2: Mobilize (preparación)
6. 6. Fase 3: Migrate (ejecución)
7. 7. CI/CD, contenedorización y automatización
8. 8. Operación, monitoreo y continuidad
9. 9. Riesgos, rollback y criterios de salida
10. 10. Anexos técnicos

1. Propósito y relación con las etapas anteriores

El objetivo de este documento es definir, de manera ordenada y técnicamente realista, cómo llevar Platzi Wallet desde su diseño lógico hacia una implementación funcional en AWS. Esta etapa no rediseña el sistema desde cero: parte de la arquitectura ya aprobada en las etapas anteriores, donde se definió una solución profesional, factible y progresiva, basada en Laravel y organizada como monolito modular cloud-ready. La comparativa previa de AWS, Azure y GCP concluyó que AWS es la plataforma más coherente para continuar, no por ser la más barata, sino por ofrecer mejor continuidad técnica, servicios administrados maduros y una ruta natural para escalar la solución sin reescribirla innecesariamente.

2. Decisión de arquitectura y estrategia de migración

La estrategia dominante para Platzi Wallet es Replatform, combinada con ajustes puntuales de Refactor de bajo riesgo. Esto significa que la aplicación conserva su núcleo funcional y su estructura modular, pero traslada responsabilidades operativas críticas a servicios gestionados: base de datos relacional en Amazon RDS, caché en ElastiCache, archivos y comprobantes en S3, mensajería asíncrona en SQS y ejecución del backend en contenedores sobre ECS Fargate. Esta decisión evita los costos y riesgos de una refactorización completa hacia microservicios distribuidos, al tiempo que elimina gran parte del mantenimiento manual de infraestructura. Para una carga académica-profesional que aspira a ser programable después, esta es la decisión más sensata.

2.1 Justificación de la elección técnica

Opción	Ventajas	Riesgos	Decisión para Platzi Wallet
Monolito simple	Implementación rápida y menor complejidad operativa.	Puede degradarse rápido si el código crece sin disciplina.	No se recomienda como destino final.
Monolito modular	Mantiene simplicidad de despliegue y orden interno por dominios funcionales.	Escala por aplicación y no por servicio independiente.	Opción seleccionada por equilibrio entre profesionalismo y factibilidad.
Microservicios completos	Escalado fino, despliegues separados y alto desacoplamiento.	Mayor complejidad de red, trazabilidad, seguridad y operación.	Se deja como evolución futura, no como punto de partida.

3. Arquitectura objetivo en AWS

Figura 1. Arquitectura objetivo de Platzi Wallet en AWS.

Lectura técnica. El cliente web/PWA consume contenido estático desde CloudFront y S3. El tráfico dinámico ingresa por un Application Load Balancer y se enruta hacia un servicio ECS Fargate que ejecuta el backend Laravel empaquetado como contenedor. La capa de datos se resuelve con RDS MySQL para persistencia transaccional, ElastiCache Redis para sesiones y cache, S3 para comprobantes y códigos QR, y SQS para desacoplar procesos asíncronos. La operación se apoya en CloudWatch, IAM roles, Parameter Store, Route 53 y ACM.

4. Fase 1: Assess (evaluación)

Propósito. Reducir incertidumbre antes de tocar producción. Aquí se inventarian componentes, se clasifican dependencias, se define la estrategia R por componente y se establece la línea base técnica y económica del sistema actual.

4.1 Inventario funcional y técnico

- Cliente web/PWA con autenticación, saldo, pagos P2P, QR, consignaciones, retiros e historial.
- Backend Laravel organizado por módulos: Auth/Usuarios, Wallet/Saldos, Payments, QR/Transferencias, Consignaciones, Retiros y Notificaciones.
- Datos principales: MySQL 8 como motor transaccional y Redis para sesiones, cache y procesos rápidos.
- Archivos y activos: comprobantes, imágenes QR, evidencias y archivos estáticos.

- Dependencias externas: Stripe o PayU para pagos; Firebase, Twilio y SendGrid para comunicación; proveedores externos como Netflix, Amazon y Xbox.

4.2 Estrategia de migración por componente

- Frontend y activos estáticos -> Replatform hacia S3 + CloudFront.
- Backend Laravel -> Replatform a contenedores en ECS Fargate, manteniendo la estructura modular.
- Base de datos MySQL -> Replatform hacia Amazon RDS for MySQL o Aurora MySQL según presupuesto y disponibilidad requerida.
- Redis -> Replatform hacia Amazon ElastiCache para reducir latencia y mantenimiento.
- Eventos y notificaciones -> Refactor ligero hacia Amazon SQS para desacoplar procesos no críticos al request principal.

4.3 Criterios de salida de la fase Assess

- Inventario validado por componente, integración y responsable.
- Matriz de riesgos inicial y dependencias críticas identificadas.
- Decisión formal de arquitectura objetivo aprobada.
- Backlog técnico de preparación y migración priorizado por oleadas.

5. Fase 2: Mobilize (preparación)

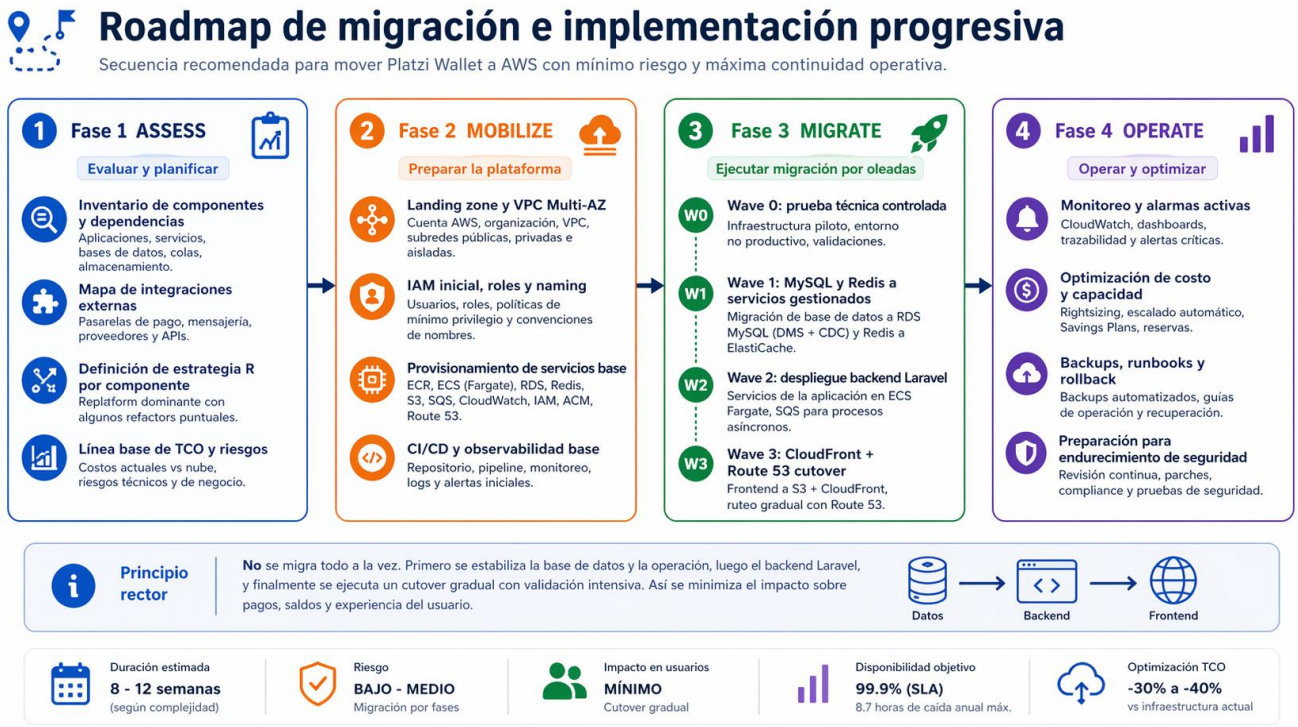


Figura 2. Roadmap de migración e implementación progresiva.

Propósito. Construir la landing zone, la red, los repositorios, la automatización y la gobernanza mínima para que la migración pueda ejecutarse sin improvisación. Esta fase no mueve todavía el núcleo del negocio; deja listo el terreno para hacerlo con control.

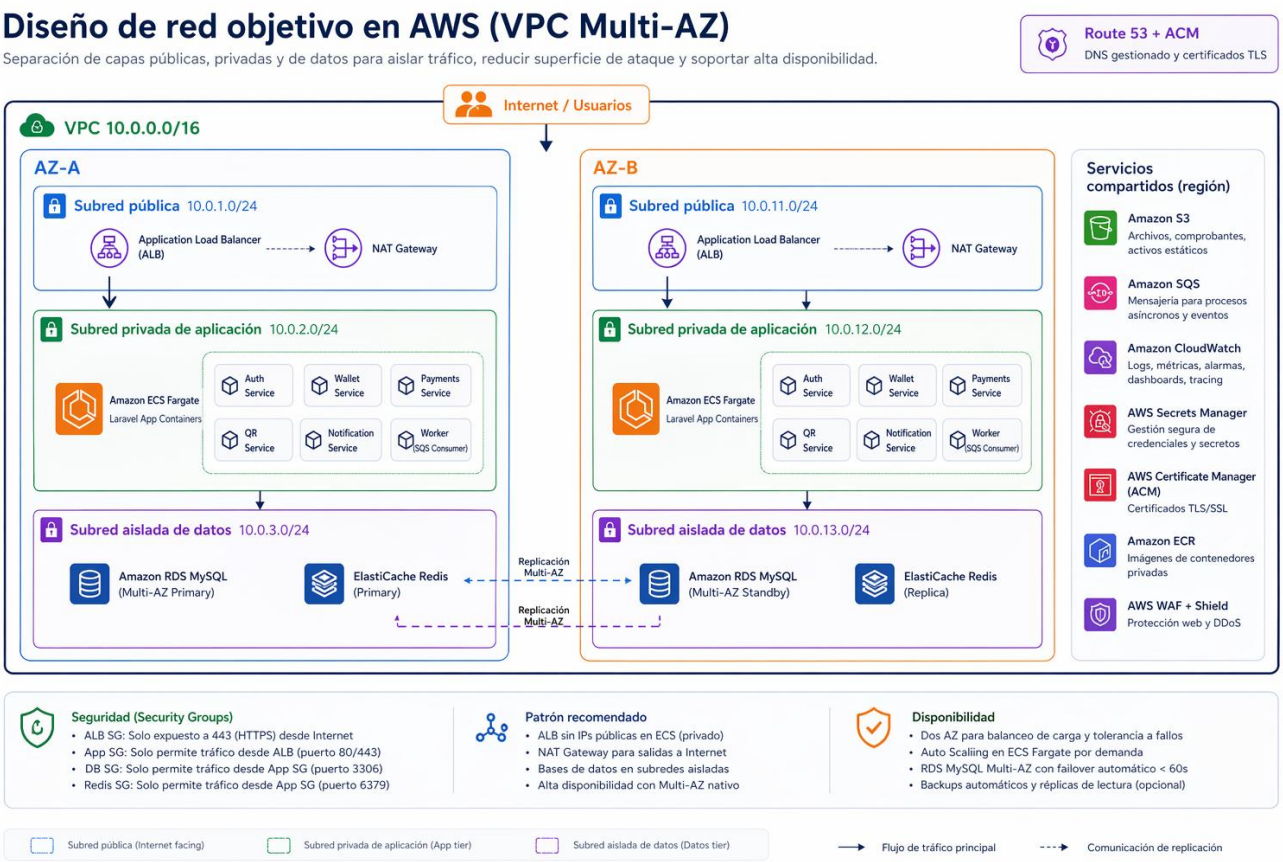


Figura 3. Diseño de red objetivo en AWS (VPC Multi-AZ).

5.1 Componentes de la landing zone

Capa	Servicio AWS propuesto	Motivo de selección
Red y entrada	VPC, subredes públicas/privadas, ALB, NAT Gateway, Route 53, ACM	Aislar capas, enrutar tráfico seguro y soportar alta disponibilidad en dos AZ.
Cómputo	Amazon ECS Fargate	Permite ejecutar contenedores Laravel sin administrar servidores y con despliegues rolling.
Persistencia	Amazon RDS MySQL	Backups, parches y alta disponibilidad administrada.
Rendimiento	Amazon ElastiCache for Redis	Reduce latencia de sesiones y cache de saldos.
Archivos	Amazon S3	Durabilidad alta para comprobantes, evidencias y archivos estáticos.
Mensajería	Amazon SQS	Desacopla notificaciones y procesos asíncronos.
Seguridad operativa	IAM Roles, Parameter Store, Security Groups	Control de acceso, secretos fuera del código y segmentación mínima.
Observabilidad	CloudWatch Logs, Metrics y Alarms	Base de operación, métricas y respuesta temprana a fallos.

5.2 Lineamientos de diseño de red y acceso

- Todo tráfico público entra por 443 hacia el ALB; el servicio ECS y la base de datos no exponen IPs públicas.
- Los contenedores Laravel viven en subredes privadas distribuidas en al menos dos AZ.

- La base de datos y Redis se ubican en subredes aisladas y solo aceptan tráfico desde la capa de aplicación.
- Los secretos se recuperan desde Systems Manager Parameter Store; no se almacenan en el repositorio ni en archivos .env persistentes de producción.
- Se definen roles específicos para ECS task execution, acceso a S3, lectura de parámetros y publicación de logs.

6. Fase 3: Migrate (ejecución)

Propósito. Mover progresivamente datos, backend, frontend y tráfico real hacia AWS sin comprometer la continuidad del servicio. La regla principal es simple: primero se migra la base que sostiene el negocio, luego la aplicación, y el corte de tráfico se hace por etapas con monitoreo intensivo y rollback definido.

6.1 Oleadas de migración recomendadas

Wave	Objetivo	Componentes	Resultado esperado
Wave 0	Prueba técnica controlada	Pipeline, repositorio ECR, servicio ECS de prueba, SQS y logs	Validar credenciales, despliegue, permisos y observabilidad.
Wave 1	Migración de datos	MySQL a RDS/Aurora MySQL y Redis a ElastiCache	Aplicación actual aún operativa, pero ya apuntando a datos gestionados.
Wave 2	Migración del backend	Backend Laravel empaquetado en Docker y desplegado en ECS Fargate	API ejecutándose en AWS con balanceo, métricas y despliegues rolling.
Wave 3	Cutover del cliente y del tráfico	S3 + CloudFront + Route 53 weighted routing	Tráfico gradual hacia AWS con validación y rollback inmediato si hace falta.

6.2 Migración de base de datos

Estrategia propuesta: Full Load + CDC con AWS DMS cuando el volumen y la continuidad operativa lo justifiquen. Para bases pequeñas o entornos de prueba, un dump inicial con validación controlada también es válido. La recomendación profesional, sin embargo, es usar DMS si existen pagos y movimientos activos durante la ventana de transición.

- Fuente: MySQL actual o instancia administrada previa; destino: Amazon RDS MySQL o Aurora MySQL.
- DMS realiza una carga completa inicial y luego replica cambios en tiempo casi real (CDC) hasta el momento del cutover.
- La validación debe incluir conteo de filas, checksums, revisión de montos agregados, saldo por usuario y pruebas de integridad del historial de movimientos.
- La aplicación antigua se mantiene operativa hasta confirmar consistencia funcional y financiera del destino.

6.3 Estrategia de cutover

- Publicar el frontend en S3 + CloudFront con certificados ACM y caché controlada.
- Crear registros en Route 53 con weighted routing para enviar inicialmente 90% del tráfico al entorno actual y 10% a AWS.
- Monitorear errores 5xx, tiempo de respuesta, latencia de pagos, integraciones externas y eventos de notificación.
- Si el comportamiento es estable, aumentar a 50/50 y luego a 100% hacia AWS.

- Solo después del 100% estable se detiene la replicación DMS y se declara completada la migración.

7. CI/CD, contenedorización y automatización

Figura 4. Pipeline CI/CD propuesto para Laravel en AWS.

7.1 Dockerfile multi-stage sugerido

```
FROM php:8.3-fpm-alpine AS base
WORKDIR /var/www/html
RUN apk add --no-cache git unzip libzip-dev oniguruma-dev icu-dev \
    && docker-php-ext-install pdo pdo_mysql intl bcmath zip
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

FROM base AS deps
COPY composer.json composer.lock ./
RUN composer install --no-dev --prefer-dist --no-interaction --optimize-autoloader

FROM base AS app
COPY . .
COPY --from=deps /var/www/html/vendor ./vendor
RUN php artisan config:cache && php artisan route:cache && php artisan view:cache
CMD ["php-fpm"]
```

7.2 Workflow de GitHub Actions sugerido

```yaml
name: ci-cd-laravel-aws
on:
  push:
    branches: [ "main" ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<account-id>:role/github-oidc-deploy
          aws-region: us-east-1
      - uses: aws-actions/amazon-ecr-login@v2
      - run: docker build -t $ECR_REPOSITORY:$GITHUB_SHA .
      - run: docker push $ECR_REGISTRY/$ECR_REPOSITORY:$GITHUB_SHA
      - uses: aws-actions/amazon-ecs-deploy-task-definition@v1
        with:
          task-definition: ecs-task-definition.json
          service: platzi-wallet-app
          cluster: platzi-wallet-prod
          wait-for-service-stability: true
```

7.3 Ejemplo mínimo de definición de tarea ECS

```
{
  "family": "platzi-wallet-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "containerDefinitions": [
    {
      "name": "laravel-app",
      "image": "<account>.dkr.ecr.us-east-1.amazonaws.com/platzi-wallet-app:<tag>",
      "essential": true,
      "portMappings": [{"containerPort": 9000, "protocol": "tcp"}],
      "environment": [
        {"name": "APP_ENV", "value": "production"},
        {"name": "CACHE_DRIVER", "value": "redis"}
      ],
      "secrets": [
        {"name": "DB_PASSWORD", "valueFrom": "/platzi-wallet/prod/db-password"}
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/platzi-wallet",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "app"
        }
      }
```

```
      }
    ]
  }
```

7.3 Ejemplo mínimo de definición de tarea ECS

Definición de tarea para ejecutar el contenedor de Laravel en ECS Fargate.

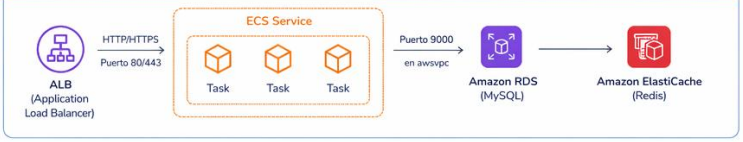
task-definition.json

```
1  {
2    "family": "platzi-wallet-app",
3    "networkMode": "awsvpc",
4    "requiresCompatibilities": ["FARGATE"],
5    "cpu": "512",
6    "memory": "1024",
7    "containerDefinitions": [
8      {
9        "name": "laravel-app",
10       "image": "%<account>.dkr.ecr.us-east-1.amazonaws.com/platzi-wallet-app:<tag>",
11       "essential": true,
12       "portMappings": [{"containerPort": 9000, "protocol": "tcp"}],
13       "environment": [
14         {"name": "APP_ENV", "value": "production"},
15         {"name": "CACHE_DRIVER", "value": "redis"}
16       ],
17       "secrets": [
18         {"name": "DB_PASSWORD", "valueFrom": "/platzi-wallet/prod/db-password"}
19       ],
20       "logConfiguration": {
21         "logDriver": "awslogs",
22         "options": {
22           "awslogs-group": "/ecs/platzi-wallet",
23           "awslogs-region": "us-east-1",
25           "awslogs-stream-prefix": "app"
26         }
27       }
29 ]   ]
```

¿Qué significa cada sección?

Sección	Descripción
family	Nombre lógico de la familia de tareas. Permite versionar y gestionar definiciones.
networkMode	awsvpc: cada tarea obtiene su propia ENI e IP dentro de la VPC.
requiresCompatibilities	Ejecuta la tarea en Fargate (sin gestionar servidores).
cpu / memory	Recursos asignados a la tarea. 512 CPU (0.5 vCPU) y 1024 MB RAM.
containerDefinitions	Configuración del contenedor que ejecutará la aplicación.
image	Imagen Docker almacenada en Amazon ECR.
portMappings	Puerto expuesto por el contenedor (9000) para Nginx / PHP-FPM.
environment	Variables de entorno no sensibles.
secrets	Variables sensibles somadas de AWS Secrets Manager / SSM Parameter Store.
logConfiguration	Envío de logs a Amazon CloudWatch Logs.

Cómo se ejecuta la tarea en ECS Fargate



Notas importantes

- La tarea usa awsvpc, por lo que cada Task tiene ENI e IP propia en la subred privada.
- El ALB enruta el tráfico hacia el puerto 9000 del contenedor.
- Las credenciales sensibles (DB_PASSWORD) se obtienen desde Secrets Manager / Parameter Store.
- Los logs se centralizan en CloudWatch para observabilidad y auditoría.

Buenas prácticas aplicadas

- Ejecutar en Fargate sin administrar servidores (infra serverless).
- Usar Secrets Manager o SSM Parameter Store para secretos.
- Centralizar logs en CloudWatch para trazabilidad.
- Asignar solo los recursos necesarios (512 CPU / 1024 Memory).
- Separación de red en subred privada con ALB en subred pública.

Nota operativa. En producción, la mejor práctica es usar OIDC o roles temporales para despliegue desde GitHub Actions, Parameter Store o Secrets Manager para secretos, y escaneo de vulnerabilidades de imagen antes de publicar en ECR.

8. Operación, monitoreo y continuidad

8.1 Observabilidad mínima obligatoria

- CloudWatch Logs por servicio y por entorno.
- Dashboards con métricas de ECS, ALB, RDS y Redis.
- Alarmas sobre 5xx, CPU, memoria, errores de conexión a base de datos, latencia y profundidad de cola en SQS.
- Trazabilidad funcional de transacciones críticas: pago creado, pago confirmado, notificación emitida, saldo actualizado.

8.2 Continuidad y respaldo

- Backups automáticos de RDS con retención y ventana definida.
- Versionado y lifecycle policies en S3 para comprobantes y QR.
- Runbook de rollback con responsables, tiempos máximos y criterios de activación.

- Pruebas periódicas de restauración en entorno controlado.

8.3 Preparación para la siguiente etapa de seguridad

- Aplicar MFA a usuarios administrativos y cuentas privilegiadas.
- Usar roles IAM de mínimo privilegio para ECS, CI/CD y acceso operativo.
- Cifrado en tránsito con HTTPS extremo a extremo y cifrado en reposo con KMS donde corresponda.
- Mantener secretos fuera del código usando Parameter Store o Secrets Manager.
- Aislar tráfico con Security Groups estrictos y revisar continuamente permisos no usados.

9. Riesgos, rollback y criterios de salida

Riesgo	Impacto	Mitigación	Rollback
Inconsistencia de datos durante la migración	Afecta saldos, movimientos y confianza del usuario.	Usar Full Load + CDC, validación por checksums y revisión de saldos agregados.	Mantener origen activo y volver a apuntar la aplicación al origen.
Degradación de rendimiento en AWS	Riesgo de latencia alta o errores 5xx.	Pruebas de carga, auto scaling inicial y monitoreo de ALB/ECS/RDS.	Reducir peso del tráfico hacia AWS mediante Route 53 weighted routing.
Fallo en integraciones externas	Pagos o notificaciones podrían quedar incompletos.	Retry controlado, logs por integración y colas asíncronas.	Aislar integración fallida y reencolar o volver temporalmente al entorno actual.
Errores de configuración o secretos	Puede provocar indisponibilidad o exposición.	Parameter Store, revisión por pares, ambientes separados y mínimo privilegio.	Revertir definición de tarea ECS y restaurar versión estable previa.

Criterios de salida para declarar exitosa la migración:

- 100% del tráfico atendido por AWS sin degradación material durante la ventana de observación.
- Conteos de filas, checksums y saldos agregados validados contra el sistema origen.
- Notificaciones, pagos y conciliaciones probadas de extremo a extremo.
- Backups, alarmas y runbooks operativos documentados.
- Equipo preparado para pasar a la etapa de endurecimiento de seguridad.

10. Conclusión

Conclusión ejecutiva. La propuesta más profesional y realista para Platzi Wallet no es rehacer el sistema entero ni simular una fintech multinacional desde el primer despliegue. La decisión correcta es consolidar una base implementable en AWS, apoyada en Laravel, contenedores administrados, datos gestionados y una migración por etapas. Este enfoque reduce riesgo, mantiene coherencia con las etapas previas del proyecto y crea una plataforma técnica suficientemente sólida para reforzar seguridad, observabilidad y crecimiento en la siguiente entrega.

11. Anexos técnicos

- Servicios AWS considerados: VPC, ALB, Route 53, ACM, ECS Fargate, ECR, RDS MySQL/Aurora MySQL, ElastiCache Redis, S3, SQS, CloudWatch, IAM, Systems Manager Parameter Store.
- Marco académico utilizado como base conceptual: arquitectura cloud para Platzi Wallet, comparativa AWS/Azure/GCP, migración en 3 fases, 7R, seguridad cloud y buenas prácticas IAM/cifrado.

- Decisión de alcance: se migra una solución monolito modular cloud-ready. La evolución a microservicios completos se considera futura y condicionada al crecimiento real del sistema.